DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 12
Docker Compose e ambientes

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Na semana passada a API ganhou uma imagem, mas subir o ambiente ainda exigia um `docker run` com várias opções e um banco que precisava existir antes. Nesta semana o ambiente inteiro (API, PostgreSQL e, quando você pedir, o pgAdmin) passa a ser descrito num arquivo versionado e sobe com um comando, na ordem certa, com os dados preservados e a configuração de cada ambiente vinda de fora do código. Você também vai entender por que o ServiceHub continua sendo um monólito, o que é um orquestrador e por que `down -v` merece respeito. A Semana 13 é a 2ª VA, que cobre o conteúdo desde a 1ª VA; das Semanas 11 e 12, priorize container × VM, camadas, tag e digest, imagem segura, rede e DNS, persistência, prontidão, configuração e segredo, monólito e microsserviços.

Conteúdo

1. Objetivos da semana
2. Aplicações multi-container
3. Monólito e microsserviços
4. Compose e orquestradores
5. Redes e DNS por nome de serviço
6. Persistência de dados
7. Iniciado não é pronto
8. Configuração e segredos
9. Paridade e perfis
10. O Compose do ServiceHub
11. Dados, pgAdmin e comandos
12. Construindo com IA
13. Glossário
14. Exercícios
15. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. explicar o que é uma aplicação multi-container e justificar por que o ServiceHub é um monólito modular, e não um conjunto de microsserviços;
2. diferenciar o Docker Compose de um orquestrador como o Kubernetes, e explicar redes, DNS por nome de serviço e as três formas de persistência (volume nomeado, bind mount e tmpfs);
3. distinguir inicialização de prontidão e usar `healthcheck` com `depends_on: condition: service_healthy`;
4. separar configuração de código e segredo de configuração, usando `.env`, `.env.example`, padrões `${VAR:-padrão}` e perfis do Spring;
5. escrever e revisar o `docker-compose.yml` da equipe e provar que a API responde na 8090 e que os dados sobrevivem a `down` e `up`.

2. Aplicações multi-container

↑ conteúdo

Uma aplicação real é um conjunto de processos que precisam se encontrar, guardar dados e subir na ordem certa. Uma **aplicação multi-container** é formada por vários containers, cada um com um papel, que juntos entregam o sistema. Exemplo genérico: uma loja virtual com um container web, um de banco de dados e um de cache.

- **Um processo por container.** Cada container roda um processo principal; se ele para, o container para, e fica claro o que falhou.
- **Ciclo de vida independente.** Atualizar a API não exige recriar o banco: cada peça tem a sua imagem e a sua versão.
- **Ligados por rede e volumes.** Os containers conversam pela rede e guardam dados em volumes, declarados junto com eles.

No ServiceHub, os serviços `api` e `postgres` sobem sempre; o `pgadmin`, só quando pedido.

ERRO COMUM EM TUTORIAIS

Colocar a aplicação e o banco no mesmo container, com um script que inicia os dois. Funciona na demonstração e quebra na primeira atualização: trocar a versão da API significa recriar o container e, junto, o banco.

3. Monólito e microsserviços

↑ conteúdo

Ter vários containers não significa ter microsserviços. São ideias diferentes. Um **monólito** é uma aplicação implantada como uma única unidade. Em **microsserviços**, a aplicação é dividida em serviços pequenos, implantados de forma independente, cada um dono dos seus dados.

Aspecto	Monólito	Microsserviços
Implantação	Um artefato, um deploy	Um deploy por serviço
Dados	Um banco para a aplicação	Cada serviço com o seu banco
Comunicação	Chamada de método, na memória	Chamadas pela rede: HTTP ou mensagens
Consistência	Uma transação do banco resolve	Vira um problema de projeto entre serviços
Operação	Poucos processos para observar	Muitos processos, rede e logs distribuídos
Ponto forte	Simples de construir, testar e depurar	Escala e evolução independentes por equipe

Nenhum estilo é melhor em absoluto: microsserviços trocam complexidade de código por complexidade de operação. Eles resolvem problemas de escala e de organização de muitas equipes, que um projeto de uma equipe pequena não tem.

O ServiceHub é um monólito modular, de propósito

Um só artefato

Um JAR, uma imagem, um deploy. Uma transação do banco garante regras como copiar o preço do serviço para a ordem no momento do pedido (RN06).

Módulos com fronteira

Usuários, serviços, ordens e avaliações têm controller, service e repository próprios. A fronteira está no código, não na rede.

O container `postgres` é um **serviço de apoio**, não um pedaço do domínio: a API continua sendo uma unidade só. Um monólito bem dividido pode ser separado depois, se um dia for preciso.

ERRO COMUM EM CÓDIGO GERADO POR IA

Propor um `user-service` e um `order-service` separados para um sistema deste porte, com o custo de rede, autenticação entre serviços e dados duplicados. Quem quiser defender outra arquitetura na banca deve registrar a decisão num ADR.

4. Compose e orquestradores

↑ conteúdo

O **Docker Compose** é a ferramenta que lê um arquivo YAML declarativo, versionado no Git, com todos os containers de um ambiente e como eles se ligam, e sobe tudo com um comando. No

Compose v2 ele é um subcomando do Docker: `docker compose`, com espaço. O antigo `docker-compose`, com hífen, ainda aparece em tutoriais.

Aspecto	`docker run`	`docker compose`
Configuração	Opções na linha de comando	Arquivo YAML versionado
Vários containers	Um comando para cada	Um comando para todos
Rede	Criada e ligada à mão	Criada automaticamente
Ordem de subida	Você controla no olho	`depends_on` declarado
Reprodutibilidade	Depende de quem digita	Igual em qualquer máquina

Orquestrar é decidir onde cada container roda, quantas cópias (**réplicas**) existem, o que fazer quando um cai e como eles se encontram. O Compose orquestra numa única máquina. Um **orquestrador** de cluster, como o **Kubernetes**, faz isso num conjunto de máquinas: recria um container que caiu, em outra máquina se preciso, e mantém o número de réplicas declarado.

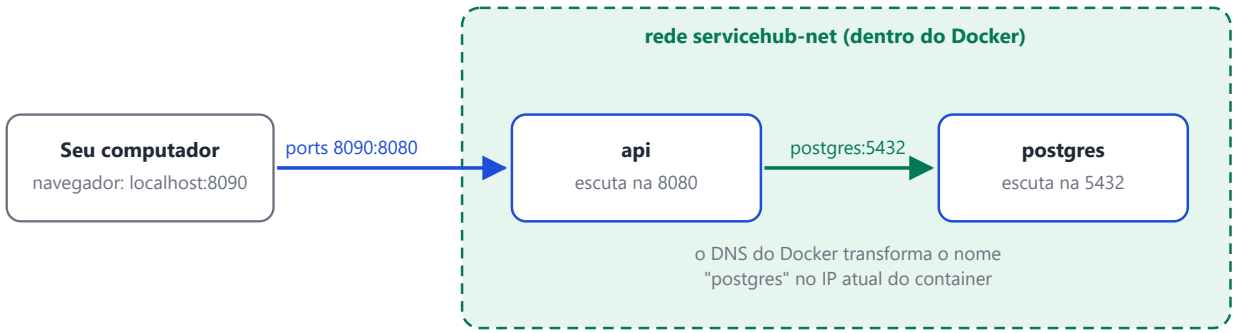
Aspecto	Docker Compose	Orquestrador (ex.: Kubernetes)
Onde roda	Uma máquina	Um cluster de várias máquinas
Uso típico	Desenvolvimento, testes, CI, demonstração	Produção com alta disponibilidade
Container caiu	`restart` reinicia na mesma máquina	Recria, em outra máquina se preciso
Escala	Manual, poucas réplicas	Réplicas declaradas; ajuste automático possível

Os dois compartilham o **estilo declarativo**: você descreve o estado desejado, e a ferramenta trabalha para chegar nele. Nesta disciplina, Kubernetes é só conceito; o ServiceHub roda com Compose no desenvolvimento, no CI e na banca.

5. Redes e DNS por nome de serviço

↑ conteúdo

Uma **rede bridge** é uma rede virtual privada entre containers da mesma máquina. Numa rede criada pelo usuário (e o Compose sempre cria uma para o projeto), o Docker oferece um **DNS interno**: cada serviço é encontrado pelo seu nome. O IP de um container muda a cada recriação; o nome do serviço não muda. Por isso a configuração usa o nome.



De fora, só se alcança o que foi publicado com `ports`; dentro da rede, os serviços se acham pelo nome e pela porta interna.

CONFUSÃO COMUM

Dentro de um container, `localhost` é o próprio container, não o seu computador. Uma API configurada com `localhost:5433` procura o banco dentro de si mesma e falha. Na rede do Compose, a URL certa é `jdbc:postgresql://postgres:5432/servicehub`: nome do serviço e porta *interna*. Publicar portas (`ports`) só é necessário para quem está fora da rede.

6. Persistência de dados

↑ conteúdo

O sistema de arquivos de um container (a camada gravável da Semana 11) some quando ele é removido. Para dados que precisam existir fora do container, o Docker oferece três mecanismos:

Tipo	Onde ficam os dados	Quando usar	Exemplo
Volume nomeado	Área gerenciada pelo Docker, identificada por um nome	Dados da aplicação, banco de dados	`pg_data` no `postgres`
Bind mount	Uma pasta ou arquivo do computador hospedeiro, escolhido por você	Configuração e código em desenvolvimento	`servers.json` do pgAdmin, somente leitura
tmpfs	Memória do hospedeiro; some quando o container para	Dados temporários ou sensíveis	Cache descartável (o ServiceHub não usa)

```
volumes:
  - pg_data:/var/lib/postgresql/data                          # volume nomeado
  - ./docker/pgadmin/servers.json:/pgadmin4/servers.json:ro  # bind mount, :ro = só leitura
tmpfs:
  - /tmp                                                      # tmpfs (exemplo genérico)
```

A diferença na sintaxe está à esquerda dos dois-pontos: um **nome** indica volume; um **caminho** (`./` ou absoluto) indica bind mount. Regra prática: banco em volume nomeado. Bind mount depende da estrutura de pastas de cada máquina, e no Windows e no macOS costuma ser mais lento.

ERRO COMUM EM TUTORIAIS

Bind mount de `./data` para o banco. Cria na pasta do projeto arquivos com dono e permissões estranhos, que atrapalham o Git e outros programas, e amarra o ambiente ao caminho de uma máquina.

7. Iniciado não é pronto

↑ conteúdo

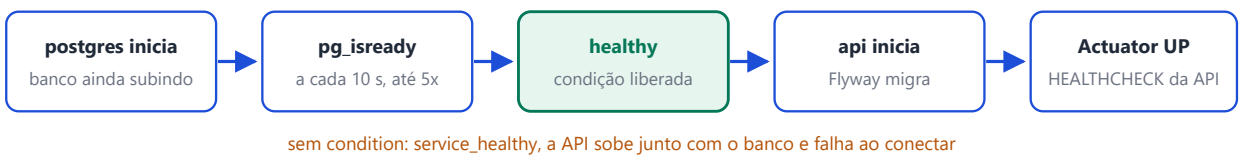
Quando um container inicia, o processo começou, mas isso não quer dizer que ele já atende. O banco pode estar lendo arquivos; a API, aplicando migrações. Há três estados diferentes:

Estado	Pergunta	Se a resposta for não
Iniciado	O processo começou?	Nada a fazer ainda: ele está subindo.
Vivo (*liveness*)	O processo ainda responde?	Reiniciar resolve.
Pronto (*readiness*)	Consegue atender requisições agora?	Espere e não mande tráfego, mas não reinicie.

ANALOGIA

Um restaurante abre as portas às 11h, mas a cozinha só serve depois de acender o forno. "Portas abertas" é iniciado; "cozinha servindo" é pronto. Quem chega às 11h01 e pede um prato precisa esperar, e ninguém resolve isso fechando o restaurante.

Um **healthcheck** é um comando que o Docker executa de tempos em tempos para marcar o container como `starting`, `healthy` ou `unhealthy`. No Compose, o `depends_on` tem três condições: `service_started` (o padrão: só espera o container existir), `service_healthy` (espera o healthcheck passar) e `service_completed_successfully` (espera o container terminar com sucesso, útil para tarefas de preparação). Para um banco, a que importa é `service_healthy`.



A ordem de subida do ServiceHub, com a saúde verificada em cada etapa.

Orquestradores como o Kubernetes separam as verificações de vida e de prontidão, e o Actuator do Spring Boot oferece grupos de saúde `liveness` e `readiness` para isso. No ServiceHub, o Compose usa o `/actuator/health` geral; os grupos ficam como conceito.

8. Configuração e segredos

↑ conteúdo

A imagem é a mesma em todo lugar; o que muda entre ambientes vem de variáveis. Essa é a ideia de quatro fatores do **Twelve-Factor App**, a metodologia já vista na Semana 11:

Fator	O que diz	No ServiceHub
III · Configuração	Guarde a configuração no ambiente, não no código.	Variáveis com padrão de dev; `.env` fora do Git
IV · Serviços de apoio	Banco, cache e fila são recursos anexados, trocáveis por configuração.	`DB_URL` aponta para qualquer PostgreSQL
X · Paridade dev/prod	Mantenha os ambientes o mais parecidos possível.	PostgreSQL 16 no Compose e nos testes
XI · Logs	Logs como fluxo de eventos na saída padrão.	Console em JSON no perfil `prod`

Um **serviço de apoio** (*backing service*) é qualquer serviço que a aplicação usa pela rede. Trocar o PostgreSQL local por um gerenciado na nuvem deve exigir só outra `DB_URL`, sem mudar uma linha de código.

Configuração não é o mesmo que segredo

Configuração comum

Porta, perfil do Spring, endereço do banco, origens do CORS, validade do token. Muda entre ambientes, mas pode aparecer em log e em documentação.

Segredo

`DB_PASS`, `JWT_SECRET`, senha do admin inicial. Dá acesso a algo; se vazou, precisa ser trocado. Nunca no Git, nunca na imagem, nunca impresso em log.

Onde o segredo vive: no `.env` da sua máquina, nos *secrets* do repositório no CI e, em produção, nas variáveis protegidas ou no **cofre de segredos** do provedor (um serviço que guarda credenciais cifradas e controla quem pode lê-las). Um segredo que entrou no histórico do Git continua lá mesmo depois de apagado do arquivo; a única correção é trocar o valor.

O que vai para o Git

Arquivo	Versionar?	O que contém
`docker-compose.yml`	Sim	Estrutura do ambiente e padrões de desenvolvimento
`.env.example`	Sim	Todas as variáveis, com valores de exemplo
`.env`	**Nunca**	Os valores reais da sua máquina ou do servidor

Arquivo	Versionar?	O que contém
`application-prod.properties`	Sim	Ajustes do perfil `prod`, sem nenhum segredo

O `.env.example` é documentação executável: quem clona copia para `.env` e troca os valores. O `.env` fica no `.gitignore` e no `.dockerignore`. Trecho do `.env.example` do ServiceHub (resumido):

```
# Porta do Postgres no seu computador (dentro do Docker é sempre 5432)
DB_PORT=5433
# Porta da API no seu computador (a bancada de testes usa 8090)
API_PORT=8090
# Segurança: troque fora da sala de aula. Mínimo de 32 caracteres; sem ele a API não sobe.
JWT_SECRET=troque-por-um-segredo-proprio-com-32-caracteres-ou-mais
```

Todo valor tem um padrão de desenvolvimento

O Compose lê automaticamente um arquivo `.env` na pasta do projeto e usa os valores para substituir as referências `${VAR}` do YAML (a **interpolação**). A forma `${VAR:-padrão}` usa o padrão quando a variável não existe ou está vazia; `${VAR:?mensagem}` faz o Compose parar com erro se ela faltar. O Spring tem a mesma ideia com outra sintaxe:

docker-compose.yml

```
POSTGRES_DB: ${DB_NAME:-servicehub}
- "${API_PORT:-8090}:8080"
```

application.properties

```
server.port=${SERVER_PORT:8080}
servicehub.jwt.secret=${JWT_SECRET:}
```

O Compose usa `:-`; o Spring, só os dois-pontos. Com padrões, o ambiente sobe num clone limpo, sem `.env`. No Spring, `JWT_SECRET` não tem padrão de propósito: sem ele, `JwtProperties` derruba a inicialização, para a API nunca rodar com segredo vazio. No Compose do projeto há um padrão de aula para ele, com a observação de trocá-lo em qualquer ambiente compartilhado.

9. Paridade e perfis

↑ conteúdo

Paridade entre desenvolvimento e produção

Quanto mais o ambiente de desenvolvimento difere do de produção, mais erros só aparecem em produção. O Twelve-Factor descreve três distâncias a reduzir: de **tempo** (o código escrito hoje chega à produção em horas ou dias, não em meses), de **pessoas** (quem escreve o código acompanha a implantação) e de **ferramentas** (mesmo banco, mesma versão e mesmos serviços de apoio em todos os ambientes). A distância de ferramentas é a que o Docker mais ajuda a fechar.

ERRO COMUM EM TUTORIAIS

H2 em memória nos testes e PostgreSQL em produção. As migrações do ServiceHub usam SQL que só o PostgreSQL entende; com H2, os testes passariam ou falhariam por motivos que não existem em produção. O projeto usa `postgres:16-alpine` no Compose e nos testes com Testcontainers (ADR-0005).

Perfis do Spring Boot

Um **perfil** é um nome, como `dev` ou `prod`, que ativa um conjunto extra de configurações. O arquivo `application-{perfil}.properties` é lido por cima do `application.properties`, e o perfil ativo é escolhido pela variável `SPRING_PROFILES_ACTIVE`. A regra de precedência que basta para esta disciplina: **variável de ambiente vence arquivo, e arquivo de perfil vence o arquivo base**. Perfil ajusta comportamento (formato e nível de log, por exemplo); segredo continua vindo do ambiente.

Texto livre (exemplo)

```
14:32:07.112 INFO [a1b2c3]
ServiceRequestService - ordem 42 aceita
```

Feito para pessoas lerem no terminal.

Estruturado (simplificado)

```
{"log.level": "INFO",
 "message": "ordem 42 aceita",
 "requestId": "a1b2c3"}
```

Feito para máquinas filtrarem: cada informação num campo com nome.

Uma ferramenta de logs busca todas as linhas de um `requestId`, ou só os erros, sem expressões regulares frágeis. O formato real usado pelo ServiceHub é o **ECS** (*Elastic Common Schema*), com mais campos que o exemplo. É configuração nativa do Spring Boot, sem dependência nova:

```
# application-prod.properties (trecho)
logging.structured.format.console=ecs
logging.level.root=INFO
logging.level.com.servicehub=INFO
logging.level.org.hibernate.SQL=WARN
```

No Compose, `SPRING_PROFILES_ACTIVE` tem padrão `prod`: `docker compose up` já sobe com logs em JSON.

Override de desenvolvimento

Para depurar, um segundo arquivo Compose é **mesclado** ao primeiro: só os campos declarados mudam, e listas como `ports` são somadas às do arquivo base.

```
# docker-compose.dev.yml
services:
  api:
    environment:
      SPRING_PROFILES_ACTIVE: dev
      JAVA_TOOL_OPTIONS: -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005
    ports:
      - "5005:5005"

docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build
```

O perfil `dev` não tem arquivo próprio no projeto: vale o `application.properties` base, que já traz log em texto com o `requestId` e SQL em DEBUG. O `JAVA_TOOL_OPTIONS` liga a depuração remota (JDWP) na porta 5005, para a IDE se conectar e parar em breakpoints; `suspend=n` faz a API subir sem esperar o depurador.

10. O Compose do ServiceHub

↑ conteúdo

O arquivo começa direto por `services:`, sem a chave `version`, que o Compose v2 ignora. Os trechos abaixo são reais; o pgAdmin aparece na seção 11.

O serviço do banco

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: servicehub-db
    environment:
      POSTGRES_DB: ${DB_NAME:-servicehub}
      POSTGRES_USER: ${DB_USER:-servicehub}
      POSTGRES_PASSWORD: ${DB_PASS:-secret}
      TZ: America/Sao_Paulo
    ports:
      - "${DB_PORT:-5433}:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-servicehub} -d ${DB_NAME:-servicehub}"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - servicehub-net
    restart: unless-stopped
```

- `postgres:16-alpine`: a mesma versão dos testes com Testcontainers (paridade).

- `POSTGRES_*`: a imagem oficial usa essas variáveis para criar banco, usuário e senha *na primeira inicialização* do volume.
- `"5433:5432"`: 5433 no seu computador, para não conflitar com um PostgreSQL instalado; 5432 dentro da rede.
- `pg_isready`: utilitário do PostgreSQL que responde se o servidor já aceita conexões; é o healthcheck do banco.
- `restart: unless-stopped`: o Docker reinicia o container se ele cair, a não ser que você o tenha parado.

O serviço da API

```
  api:
    build: .
    image: servicehub-api:latest
    container_name: servicehub-api
    env_file:
      - path: .env
        required: false
    environment:
      SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE:-prod}
      DB_URL: jdbc:postgresql://postgres:5432/${DB_NAME:-servicehub}
      DB_USER: ${DB_USER:-servicehub}
      DB_PASS: ${DB_PASS:-secret}
      JWT_SECRET: ${JWT_SECRET:-troque-este-segredo-de-desenvolvimento-com-32-bytes}
      # ... JWT_EXPIRATION_SECONDS, SERVICEHUB_ADMIN_*, CORS_ALLOWED_ORIGINS, TZ
    ports:
      - "${API_PORT:-8090}:8080"
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - servicehub-net
    restart: unless-stopped

networks:
  servicehub-net:
    driver: bridge

volumes:
  pg_data:
```

(Resumido: as variáveis indicadas no comentário estão no arquivo real.)

- `build: .` usa o Dockerfile multi-stage da Semana 11; `image` dá nome à imagem construída.
- `env_file` com `required: false`: se houver `.env`, suas variáveis chegam ao container; se não houver, valem os padrões. Quando a mesma variável aparece nos dois, a de `environment` prevalece.

- `DB_URL` aponta para `postgres:5432`, nome do serviço e porta interna (seção 5).
- `"8090:8080"`: a porta que a bancada de testes e os critérios do Mapa esperam.
- `depends_on` com `condition: service_healthy`: a API só sobe com o banco pronto (seção 7). O HEALTHCHECK da própria imagem marca a API como `healthy` quando o Actuator responde `UP`.
- `networks` e `volumes` no fim declaram a rede e o volume nomeado usados pelos serviços.

O MESMO ARQUIVO EM TRÊS LUGARES

O pipeline de CI do projeto roda `docker compose up -d --build`, espera o container da API ficar `healthy` e faz um `curl` em `http://localhost:8090/api/v1/info`. O mesmo arquivo serve ao seu notebook, ao pipeline (Semana 14) e à banca.

11. Dados, pgAdmin e comandos

↑ conteúdo

down e down -v: uma letra de diferença

docker compose down

Para e remove os containers e a rede. O volume `pg_data` fica: no próximo `up`, usuários, serviços e ordens continuam lá.

docker compose down -v

Remove também os volumes nomeados. O banco é apagado, e o Flyway recria o schema do zero no próximo `up`. Use só quando quiser apagar os dados de propósito.

ARMADILHA CLÁSSICA

As variáveis `POSTGRES_*` só valem na criação do volume. Se você muda `DB_PASS` no `.env` depois, a API passa a mandar a senha nova, mas o banco guardado no volume continua com a antiga, e a autenticação falha. Outra: volumes pertencem a um **projeto** Compose, cujo nome padrão é o da pasta; com outro nome de projeto (opção `-p`), o volume é outro e o banco parece "vazio".

pgAdmin no profile tools

Um **profile** do Compose marca serviços que só sobem quando pedidos. O pgAdmin, interface web para inspecionar o banco, é uma ferramenta de apoio: não deve pesar no ambiente de todo dia. Trecho do serviço (resumido):

```
  pgadmin:
    image: dpage/pgadmin4
    profiles: ["tools"]
    environment:
      PGADMIN_CONFIG_SERVER_MODE: "False"
      PGADMIN_CONFIG_MASTER_PASSWORD_REQUIRED: "False"
    volumes:
      - ./docker/pgadmin/servers.json:/pgadmin4/servers.json:ro
    ports:
      - "127.0.0.1:${PGADMIN_PORT:-8091}:80"
```

Com `docker compose --profile tools up -d`, o pgAdmin abre em `http://localhost:8091` em modo desktop, sem login, já com dois servidores cadastrados pelo `servers.json`: o banco do projeto (`postgres:5432`, pela rede do Compose) e o "banco do aluno" (`host.docker.internal:5433`). As senhas vêm de um arquivo *pgpass* que o próprio serviço grava ao iniciar. A porta fica presa a `127.0.0.1` porque o modo sem login só é aceitável na máquina local.

Os comandos do dia a dia

```
docker compose up -d --build         # sobe tudo e reconstrói a imagem da API
docker compose ps                    # estado de cada serviço, com (healthy)
docker compose logs -f api           # log da API em tempo real
docker compose exec api env          # variáveis que chegaram ao container
docker compose exec postgres psql -U servicehub -d servicehub
docker compose --profile tools up -d  # inclui o pgAdmin
docker compose down                  # para tudo e mantém pg_data
docker compose down -v               # para tudo e APAGA os dados
```

Algo não subiu? Comece por `ps` e `logs`. Variável errada? `exec api env` mostra o que o container recebeu. Porta 8090 ocupada? `API_PORT` no `.env` resolve sem editar o YAML.

12. Construindo com IA

↑ conteúdo

Um `docker-compose.yml` gerado por IA quase sempre "sobe". O problema é o que ele esconde: a API que sobe antes do banco e falha de vez em quando, a senha real versionada, o banco que some no primeiro `down -v`. Os quatro movimentos, nesta semana:

Movimento	Nesta semana
Especificar	API só sobe com o banco `healthy`; todo valor com padrão de dev; `.env` opcional; volume nomeado; pgAdmin só no profile `tools`.
Gerar	Pedir o plano (serviços, rede, volume, variáveis e seus padrões) antes do YAML.
Revisar	Sem `version`, imagem com versão fixa, `DB_URL` com `postgres:5432`, `condition` presente, nenhuma senha real no YAML.
Provar	Num clone limpo, sem `.env`: `up -d --build`, os dois serviços `(healthy)`, `/api/v1/info` na 8090 e dados vivos depois de `down` e `up`.

Prompt fraco e prompt bom

Fraco

```
faz um docker compose com
spring boot e postgres
```

Tende a vir com `version: '3.9'`, `postgres:latest`, `depends_on` sem condição, senha no YAML e `localhost` na URL do banco.

Bom

Parte do Dockerfile da semana anterior, fixa as versões, nomeia os serviços, exige healthcheck e `service_healthy`, diz como a configuração entra (`${VAR:-padrão}` e `.env` opcional), proíbe segredo no YAML, pede plano antes e define a prova: clone limpo, 8090 e dados que persistem.

O prompt desta semana

Continua o da Semana 11: a API já tem o Dockerfile multi-stage com HEALTHCHECK. Troque o que estiver entre <>:

```
Contexto: somos uma equipe construindo a API ServiceHub em Java 21 e
Spring Boot 4.1, com PostgreSQL 16 e Flyway. Na raiz do repositório já
existe um Dockerfile multi-stage (runtime eclipse-temurin:21-jre-alpine,
porta 8080, HEALTHCHECK em /actuator/health). A API lê do ambiente:
DB_URL, DB_USER, DB_PASS, JWT_SECRET (obrigatória, mínimo de 32 bytes),
JWT_EXPIRATION_SECONDS, SERVICEHUB_ADMIN_EMAIL, SERVICEHUB_ADMIN_PASSWORD,
CORS_ALLOWED_ORIGINS e SPRING_PROFILES_ACTIVE. Existe
application-prod.properties com logging.structured.format.console=ecs.
Seguem o Dockerfile e o application.properties: <cole aqui>.

Objetivo: criar docker-compose.yml e .env.example na raiz para subir API
e banco com um comando.

Restrições:
1. Sem a chave "version". Serviços "postgres" e "api" numa rede bridge
   nomeada servicehub-net.
2. postgres: image postgres:16-alpine; POSTGRES_DB, POSTGRES_USER e
   POSTGRES_PASSWORD vindos de ${DB_NAME:-servicehub}, ${DB_USER:-servicehub}
   e ${DB_PASS:-secret}; porta "${DB_PORT:-5433}:5432"; volume nomeado
   pg_data em /var/lib/postgresql/data; healthcheck com pg_isready
   (interval 10s, timeout 5s, retries 5); restart unless-stopped.
3. api: build . ; env_file .env com required: false; DB_URL
   jdbc:postgresql://postgres:5432/${DB_NAME:-servicehub} (nunca localhost);
   SPRING_PROFILES_ACTIVE com padrão prod; toda variável com padrão de
   desenvolvimento ${VAR:-padrão}; porta "${API_PORT:-8090}:8080";
   depends_on postgres com condition: service_healthy.
4. Nenhuma senha ou segredo real no YAML: só padrões de desenvolvimento.
   O .env.example lista TODAS as variáveis com valores de exemplo; .env
   continua no .gitignore e no .dockerignore.
5. NÃO use postgres:latest nem outra versão, docker-compose com hífen,
   bind mount para os dados do banco, links nem container separado para
   cada módulo da API. NÃO altere código Java, pom.xml nem o Dockerfile.

Antes do YAML, apresente um plano curto: serviços, rede, volume, ordem de
subida e a tabela de variáveis com seus padrões. Espere minha confirmação.
Depois entregue os dois arquivos completos, comentados.

Critério de aceite: num clone limpo, sem .env, docker compose up -d --build
sobe os dois serviços; docker compose ps mostra ambos (healthy);
GET http://localhost:8090/api/v1/info responde 200; um usuário cadastrado
continua existindo depois de docker compose down e docker compose up -d.
```

O que a IA costuma errar nesta etapa

O que aparece no YAML gerado	O que o ServiceHub faz
`version: '3.9'` no topo	Sem a chave: o Compose v2 a ignora
`postgres:latest` ou `postgres:15`	`postgres:16-alpine`, a versão dos testes
`depends_on` só com o nome do serviço	healthcheck e `condition: service_healthy`
`DB_URL` com `localhost:5433` na API	`postgres:5432`, nome e porta internos

O que aparece no YAML gerado	O que o ServiceHub faz
`env_file: .env` obrigatório	`required: false` e padrões `${VAR:-...}`
Senha real escrita no YAML versionado	Valores reais só no `.env`, fora do Git
`down -v` sugerido para "limpar"	`down`; o `-v` só para apagar o banco

Checklist de revisão

☐ O arquivo começa em `services:`, sem `version`?

☐ A imagem do banco é `postgres:16-alpine` e os dados estão num volume nomeado?

☐ O banco tem healthcheck e a API tem `depends_on` com `condition: service_healthy`?

☐ A `DB_URL` da API usa `postgres:5432`?

☐ Todo `${VAR}` tem padrão, e o ambiente sobe sem `.env`?

☐ Nenhum valor real de senha ou segredo está no YAML ou no `.env.example`?

☐ A porta publicada da API é 8090?

Prova

Faça num clone limpo do repositório, sem `.env`:

```
docker compose up -d --build
docker compose ps                          # api e postgres (healthy)
curl http://localhost:8090/api/v1/info     # no PowerShell, use curl.exe

# cadastre um usuário pelo Swagger em http://localhost:8090/docs
# (POST /api/v1/auth/register) e conte os usuários:
docker compose exec postgres psql -U servicehub -d servicehub -c "select count(*) from users"

docker compose down
docker compose up -d
docker compose exec postgres psql -U servicehub -d servicehub -c "select count(*) from users"
# a contagem é a mesma: os dados persistiram no volume pg_data
```

Para ver a diferença, repita com `down -v`: a contagem volta a ter só o admin criado na inicialização. Registre no PR o pedido feito à IA, o que você corrigiu e a saída da prova.

13. Glossário

↑ conteúdo

Serviço	No Compose, a definição de um container (imagem, variáveis, portas, volumes); seu nome vira nome de DNS na rede.
Monólito modular	Aplicação implantada como unidade única, com módulos de fronteira clara no código.
Microsserviços	Estilo em que a aplicação é dividida em serviços implantados de forma independente, cada um dono dos seus dados.
Orquestrador	Ferramenta que decide onde os containers rodam, quantas réplicas existem e o que fazer quando caem, num cluster.
Rede bridge	Rede virtual privada entre containers da mesma máquina, com DNS por nome de serviço.
Volume nomeado	Área de dados gerenciada pelo Docker, que sobrevive à remoção do container (diferente do bind mount, que monta uma pasta do hospedeiro).
Prontidão	Capacidade de atender requisições agora; diferente de apenas ter iniciado.
Healthcheck	Comando periódico que marca o container como starting, healthy ou unhealthy.
Serviço de apoio	Recurso usado pela rede (banco, fila, cache), trocável por configuração.
Segredo	Configuração que dá acesso a algo (senha, chave, token); se vazar, precisa ser trocada.
Perfil	Nome que ativa um conjunto extra de configurações no Spring Boot, como `prod`.

14. Exercícios

↑ conteúdo

1. O ServiceHub roda em dois containers. Isso faz dele uma arquitetura de microsserviços? Justifique com as definições de monólito, microsserviços e serviço de apoio, e cite uma situação em que valeria separar um módulo em serviço próprio.
2. Uma equipe configurou a API com `DB_URL=jdbc:postgresql://localhost:5433/servicehub` dentro do Compose, e a API não conecta. Explique o erro com o conceito de rede e DNS e corrija a URL.
3. Para cada caso, escolha volume nomeado, bind mount ou tmpfs e justifique: (a) os dados do PostgreSQL; (b) um arquivo de configuração do pgAdmin editado no repositório; (c) arquivos temporários que não podem ir para o disco.
4. A API sobe e cai com "Connection refused" uma vez em cada três tentativas, mas o `docker-compose.yml` tem `depends_on: [postgres]`. Explique a causa usando a diferença entre iniciado e pronto e mostre o trecho YAML que corrige.

5. Classifique como configuração comum ou segredo: `API_PORT`, `JWT_SECRET`, `SPRING_PROFILES_ACTIVE`, `DB_PASS`, `CORS_ALLOWED_ORIGINS`, `SERVICEHUB_ADMIN_PASSWORD`. Onde vive cada segredo no notebook, no CI e em produção?

6. Um colega trocou `DB_PASS` no `.env` e agora a API não autentica no banco. O que aconteceu? Dê duas formas de resolver e diga o custo de cada uma.

7. O que muda nos logs entre o Compose padrão e o override `docker-compose.dev.yml`? Por que essa diferença está em perfil, e não no código?

8. Revise o trecho gerado por IA e liste os problemas:

```
version: '3.9'
services:
  db:
    image: postgres:latest
    environment: { POSTGRES_PASSWORD: SenhaReal@2026 }
    volumes:
      - ./data:/var/lib/postgresql/data
  api:
    build: .
    environment: { DB_URL: "jdbc:postgresql://localhost:5432/servicehub" }
    depends_on: [db]
```

9. Escreva um prompt para pedir à IA que adicione o pgAdmin ao Compose da equipe só no profile `tools`, sem login, acessível apenas pela máquina local. Diga o que você conferiria na resposta.

15. Para estudar

↑ conteúdo

Leitura disponível na Minha Biblioteca, pelo acesso institucional:

- MONTEIRO, E. R. et al. *DevOps*. Grupo A, 2021. Capítulos "Uso do Docker para microsserviços" e "Gerenciando o ciclo de vida de contêineres".

Referências abertas:

- *The Twelve-Factor App*, fatores III (Configurações), IV (Serviços de apoio), X (Paridade entre desenvolvimento e produção) e XI (Logs): `12factor.net` (há versão em português).
- Documentação oficial: Docker, *Compose file reference* (`docs.docker.com/reference/compose-file`); Spring Boot, *Externalized Configuration* e *Structured Logging* (`docs.spring.io/spring-boot`).
- No repositório do ServiceHub: `docker-compose.yml`, `docker-compose.dev.yml`, `.env.example`, `docker/pgadmin/servers.json` e o ADR-0005.